Um agente de QA que aprende as regras do negócio — e nunca deixa de testá-las.

Proposta para transformar conhecimento de regra de negócio, hoje concentrado no time, em testes de regressão gerados, mantidos e auditáveis por um agente de IA acionável em um comando.

Documento
`Plano de produto — protótipo de manual — para decisão de investimento`

Escopo do MVP
`1 projeto web · regra de negócio → teste E2E`
Interface
`CLI (qa-agent)`
Stack
`Node.js/TypeScript + Cypress`
Status
`Conceito refinado, pronto para build`

I. O problema

Nosso maior risco de qualidade não é código malfeito — é conhecimento que existe só na cabeça de quem já resolveu aquele caso antes. Regras de negócio se acumulam em anos de tasks, PRs e conversas, espalhadas por múltiplos projetos sem um padrão de arquitetura em comum.

> *Quando uma regra vigente não está escrita em lugar nenhum consultável, ela só é lembrada até a pessoa certa sair de férias — ou do time.*

Regressão silenciosa
Uma feature nova reintroduz um comportamento que já foi corrigido — porque a regra que evitava isso nunca ficou registrada de um jeito que o próximo engenheiro pudesse consultar.

Fragmentação multi-projeto

A mesma regra pode atravessar frontend e backend, em repositórios diferentes, sem padrão comum — o que dificulta até saber que a regra existe.

QA como gargalo caro
Cobertura de regressão depende de tempo de engenheiro sênior revalidando manualmente o que o time já sabe de cabeça — não escala.

A dor mais urgente, validada em refinamento com o time: **não é falta de teste em geral — é falta de teste de regressão amarrado à regra de negócio**. O valor só é percebido quando o teste regressivo é de fato gerado e mantido, não com um repositório de regras passivo para consulta.

II. A tese do produto

Não é uma ferramenta única — é um agente de QA autônomo, acionável pelo engenheiro no terminal, que entende a regra, escreve o teste, roda e presta contas do que cobriu.

1. Entende
Busca a regra de negócio na task, valida com o engenheiro responsável em uma sessão de refinamento — nunca decide sozinho.

2. Codifica
Gera o teste de regressão de verdade, no framework que o projeto já usa — não é um sugestor de texto, é um agente de codificação.

3. Presta contas
Roda o teste e devolve um relatório explícito: o que da regra foi coberto, o que não foi, o que ficou incerto.

Por que CLI, e não um portal: o time já vive em terminal e IDE. O mesmo binário roda local, no comando do engenheiro, e na esteira do GitHub Actions — sem duplicar lógica entre uso manual e automatizado, e sem tirar ninguém do fluxo de trabalho para "abrir um sistema à parte".

1	interface — o CLI, reaproveitado por engenheiro e pipeline
5	módulos desenhados, entregues em cortes verticais que já provam valor
0%	de regra confirmada sem validação humana

III. Como funciona

O núcleo do sistema não tenta "entender o projeto inteiro" — inviável sem padrão de arquitetura comum entre repositórios. Ele entende um recorte por vez: a feature X, conforme a task Y, tocando os arquivos Z — através de três camadas de confiança.

Camada	Confiança	Descrição
A	Alta	A task na ferramenta de gerenciamento de projeto. Histórias e critérios de aceite são a fonte primária, mais barata e mais confiável de regra de negócio.
B	Média	O código como evidência — nunca fonte de verdade — usado para linkar regra ↔ implementação, dando rastreabilidade task → arquivos → PR.
C	Baixa	Inferência: na ausência de task ou doc, a IA infere a partir do código, mas marca explicitamente como "inferido, não confirmado" até validação humana.

O mecanismo de confiabilidade: nenhuma regra nasce sozinha

Toda regra nova — venha do bootstrap inicial de um projeto ou de uma task do dia a dia — passa por uma **sessão de refinamento com o engenheiro responsável** antes de virar fonte de verdade. A IA propõe e extrai; o engenheiro valida, ajusta ou rejeita, em uma conversa — nunca um "aprovar" de tela única.

A sessão roda em **lote**: todas as regras candidatas de uma feature de uma vez, não pergunta por pergunta — evita gargalo e dá ao engenheiro visão de conjunto para notar contradições entre as próprias regras da mesma entrega.

Conhecimento técnico do projeto: a "skill"

Além da regra de negócio (o quê o sistema deve fazer), o agente também aprende como aquele projeto especificamente é construído e testado — convenção de estrutura de teste, padrão de mock, nomenclatura.

	Regra de negócio	Skill técnica do projeto
Responde	O quê o sistema faz	Como este projeto é construído/testado
Volatilidade	Alta — muda com frequência	Baixa — convenção muda raramente

Governança	Sessão de refinamento + detecção de conflito	Mesma sessão + revalidação por âncora técnica
Nasce de	Task → código → inferência	Observação do código existente no bootstrap

IV. Guardrails de custo de IA

Um sistema que "entende regra de negócio" pode virar caro rápido se mandar contexto demais para o modelo a cada chamada. O desenho já nasce com tetos explícitos, não como ajuste posterior.

Retrieval seletivo — nunca o repo/board inteiro
Cada operação recebe só o subgrafo de regra relevante à feature — não o histórico completo do projeto nem do board de tasks.

Orçamento, não contagem — teto por tokens, não por item
Em vez de "top-10 regras", um orçamento máximo de tokens por operação — corta as menos relevantes até caber, começando conservador e calibrado com dado real do piloto.

Higiene de dado — regra revogada não entra na conta
Regras substituídas saem da comparação ativa por padrão — sem isso, o conjunto comparado a cada nova regra só cresceria com a idade do projeto.

Rede de segurança — teto agregado mensal
Além do limite por operação, um teto diário/mensal no orquestrador corta ou alerta antes que um caso não previsto consuma o orçamento de um mês inteiro de uma vez.

Efeito prático: o consumo de token não cresce junto com a maturidade da base de regras — cresce com o volume de trabalho novo, que é o mesmo custo que já existiria de qualquer forma.

V. Escopo do MVP

Um corte vertical fino: prova de que dá pra ir de "regra de negócio validada" a "teste E2E regressivo executável, com relatório de cobertura" — nada além disso precisa funcionar ainda.

Dentro	Fora — evolução planejada
• Um projeto web puro (Angular ou Next) — sem WebView nesta fase • Task → sessão de refinamento → regra confirmada • Geração (não incremento) de teste E2E em Cypress • Relatório de cobertura da regra: coberto vs. não coberto • CLI funcional, com poucos comandos	• Canal WebView/SDK próprio • Bootstrap completo, base central cross-projeto, detecção de conflito • Skill técnica por projeto • Incremento de teste existente • Testes unitários, massa de dados, dashboard, orquestrador de triagem

> *Critério de aceite: 1 task real → sessão de refinamento → teste E2E executável e correto → relatório explícito do que da regra ficou coberto.*

VI. Manual de uso — protótipo

Como a experiência se parece na prática, do ponto de vista do engenheiro, do primeiro comando ao relatório final.

```
$ qa-agent gerar-testes --task=XPTO-2231

buscando XPTO-2231 na ferramenta de gerenciamento de projeto…
✓ critério de aceite encontrado (Camada A)

regra candidata detectada:
  "desconto máximo de 20% sobre o pedido sem aprovação de supervisor"

── sessão de refinamento ──────────────────────────
? Essa regra vale pra todo tipo de cliente, ou só varejo?
› vale pra todos, exceto contas corporativas (regra própria, TASK-88)
? No limite exato de 20%, conta como "acima" ou "dentro"?
› dentro — só acima de 20% exige aprovação
✓ regra confirmada e salva em .qa-context/regras/desconto-maximo.json

gerando teste E2E (Cypress, convenção já usada no projeto)…
✓ cypress/e2e/pedido/desconto-maximo.cy.ts criado
executando… 1 passed
```

O relatório final não para em "passou" — mostra explicitamente o que da regra ficou coberto:

✓	Desconto de 20% aplicado sem exigir aprovação (caminho principal)
✓	Desconto de 20,01% bloqueado sem aprovação de supervisor
△	Fluxo de aprovação do supervisor concedendo o desconto acima do limite — não coberto, exige mock de permissão de supervisor
△	Comportamento para conta corporativa (regra distinta, TASK-88) — não coberto neste teste

A decisão de aceitar essa cobertura ou pedir incremento é do engenheiro — o agente nunca declara "cobertura suficiente" sozinho.

VII. Roadmap de execução do MVP

Oito fases, cada uma desbloqueando a próxima — sem construir infraestrutura que o MVP ainda não precisa.

1. **Esqueleto do CLI** — projeto Node.js/TypeScript, comando *qa-agent gerar-testes --task=X*.
2. **Camada A — task** — CLI como cliente MCP da ferramenta de gerenciamento de projeto já disponível; busca critérios de aceite.
3. **Adapter de LLM** — interface model-agnostic, com uma implementação real — desacoplada de provedor específico desde o início.
4. **Sessão de refinamento** — extração de regra candidata, sessão em lote no CLI, confirmação salva localmente.
5. **Servidor de contexto mínimo** — mcp-project-context servindo a regra confirmada, sem índice de busca ainda.
6. **Geração do teste Cypress** — a partir da regra confirmada e de exemplos de teste já existentes no projeto.
7. **Execução + relatório de cobertura** — roda o teste gerado e compara contra o enunciado completo da regra.
8. **Validação ponta a ponta** — fluxo completo com uma task real do projeto piloto — critério de aceite do MVP.

VIII. Riscos mapeados

Nada aqui foi ignorado — foi endereçado no desenho ou fica explicitamente como próxima decisão, não como ponto cego.

Risco	Como é endereçado	Status
Custo de IA cresce com a maturidade da base	Teto de contexto por operação + rede de segurança agregada mensal	Endereçado
IA "aprende errado" uma regra ou convenção do projeto	Nenhuma regra ou skill vira fonte de verdade sem sessão de refinamento humana	Endereçado
Duas edições concorrentes de regra em paralelo	Conflito textual resolvido nativamente pelo controle de versão; conflito semântico pego por sync reforçado antes do commit final	Endereçado
Teste passa mas não cobre a regra inteira	Relatório de cobertura explícito, comparando teste contra o enunciado completo	Endereçado
Reinventar infraestrutura de massa de dados que já existe	Módulo 3 reaproveita o portal de massas já em uso (geração + seed de ambiente) via wrapper MCP fino; lacunas viram pedido aos mantenedores	Endereçado
Canal WebView pode se comportar diferente do canal web direto	Identificado e escopado como evolução planejada pós-MVP, não ignorado	Em observação
Valor concreto do orçamento de token ainda não calibrado	Ponto de partida conservador, ajustado com dado real do piloto	Em observação

IX. Evolução planejada além do MVP

O MVP prova o corte fino. Nenhuma capacidade descartada — só sequenciada.

Incremento de teste
Ajustar teste existente em vez de só gerar novo, aproximando-se de um agente de codificação completo.

Canal WebView/SDK
Cobrir também o canal de comunicação via SDK próprio, não só o acesso web direto.

Base central cross-projeto
Repositório único consolidando regras de todos os projetos, com detecção de conflito entre times.

Skill técnica por projeto
Convenções técnicas aprendidas e reaproveitadas, reduzindo ainda mais custo de token por geração.

Massa de dados (Módulo 3) — reaproveita infra existente
Não construímos do zero: integração com o portal de massas já mantido pelo time, via wrapper MCP fino.

Painel de rastreabilidade (Módulo 5)
Cobertura por regra de negócio, não só por linha — visível para além do engenheiro individual.

X. Glossário rápido

Regra de negócio	Fato de domínio versionado — o que o sistema deve fazer — com origem, camada de confiança e histórico de substituição.
Sessão de refinamento	Conversa entre o agente e o engenheiro responsável, em lote, que valida toda regra ou skill antes de virar fonte de verdade.
Camadas A/B/C	Hierarquia de confiança da regra: task (alta), código como evidência (média), inferência da IA (baixa, sempre marcada como tal).
Skill técnica	Conhecimento procedural do projeto — como ele testa, nomeia, estrutura — complementar à regra de negócio.
Relatório de cobertura	Comparação entre o teste gerado e o enunciado completo da regra, listando o que foi e o que não foi exercitado.

Próximo passo

O que precisamos para começar o MVP

Escolher o piloto
Um projeto Angular ou Next, com tasks bem preenchidas, escopo pequeno.

Acesso ao MCP de tasks
Confirmar o que a ferramenta de gerenciamento de projeto já expõe hoje, para não duplicar capacidade.

Tempo de build
Oito fases sequenciais, cada uma com entrega verificável — sem infraestrutura antecipada.

Documento vivo — QA Inteligente — atualizado 2026-08-03.